

Build Costs Calculation Documentation

1 Introduction

The `build_costs.py` script calculates the construction costs for a transmission line project. It computes costs for conductors, structures, and converters, applies terrain adjustments via weighted miles, incorporates contingency factors, and calculates both regulatory (AFUDC) and societal (present value) perspectives.

2 Method Overview

2.1 Purpose

This script calculates the total build cost of a transmission line project by:

1. Loading project technical details and constructing category via `calculation_utils.build_category`
2. Loading base costs from `10_project_category_build_costs.yaml` for that category
3. Adjusting costs for terrain complexity using `weighted_miles.calculate_weighted_miles()`
4. Applying per-component contingency factors (conductor, structure, converter)
5. Loading AFUDC setup via `financial_utils.load_afudc_setup`; if enabled, calculating AFUDC capitalized cost using timing patterns for regulatory perspective
6. Computing present value of build costs (spread evenly over construction period) for societal perspective
7. Writing results via `CTCCOutputManager.add_build_costs` and `write_batch_summary`

2.2 Calculation Approach

The build cost calculation follows this structure:

$$\text{Conductor Cost} = (\text{Variable Cost/Mile} \times \text{Weighted Miles}) + \text{Fixed Cost} \quad (1)$$

$$\text{Structure Cost} = \text{Variable Cost/Mile} \times \text{Weighted Miles} \quad (\text{if not reconductoring}) \quad (2)$$

$$\text{Converter Cost} = \text{Fixed Cost} \times \text{Number of Converters} \quad (\text{if not reconductoring}) \quad (3)$$

$$\text{Total Base Cost} = \text{Conductor} + \text{Structure} + \text{Converter} \quad (4)$$

$$\text{Conductor with Conting.} = \text{Conductor} \times (1 + \text{conductor_contingency}) \quad (5)$$

$$\text{Structure with Conting.} = \text{Structure} \times (1 + \text{structure_contingency}) \quad (6)$$

$$\text{Converter with Conting.} = \text{Converter} \times (1 + \text{converter_contingency}) \quad (7)$$

$$\text{Total with Contingencies} = \text{sum of above three} \quad (8)$$

For reconductoring projects, structure and converter costs are set to zero since existing infrastructure is reused.

2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, AC/DC, capacity, conductor type, etc.)
- Physical details (terrain distribution)
- Build cost parameters from `10_project_category_build_costs.yaml` (by category from `build_category_string`)
- Per-component contingency percentages (`conductor_contingency`, `structure_contingency`, `converter_contingency`) from financing
- Financing parameters (WACC, inflation); AFUDC from `load_afudc_setup` (timing patterns, AFUDC rate)

Outputs:

- Nominal costs (with contingencies)
- AFUDC capitalized costs (regulatory perspective)
- Present value costs (societal perspective)
- Component-level costs (conductor, structure, converter)

2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` - `load_project_technical_details`, `load_physical_details`, `load_contingencies`, `load_financing_details`, `load_cost_timing_patterns`, `load_afudc_config`, `get_project_data_raw`, `get_financing_data_raw`
- `weighted_miles.py` - `calculate_weighted_miles` for terrain-adjusted miles
- `calculation_utils.py` - `build_category_string` for project category
- `financial_utils.py` - `calculate_present_value`, `calculate_afudc_capitalized_cost`, `load_afudc_setup`, `calculate_construction_start_year`, `validate_discount_rate`
- `path_config.py` - `YAMLS_DIR` for build costs YAML
- `smart_output.py` - `CTCCOutputManager` (writes CSV or JSON via `add_build_costs`, `write_batch_summary`)

3 Key Functions

3.1 `load_costs(category, total_miles, number_of_converters, contingencies, reconductoring)`

Loads and calculates build costs for the specified project category.

Parameters:

- `category` (str): Project category identifier in format "ConstructionType/AC_DC/CapacityMW/ConductorType"
- `total_miles` (float): Total miles of transmission line
- `number_of_converters` (int): Number of converters needed (0 for AC)
- `contingencies` (dict): Dictionary with keys "conductor_contingency", "structure_contingency", "converter_contingency"
- `reconductoring` (bool): Whether this is a reconductoring project

Returns: A `BuildCosts` dataclass with fields: `total_cost`, `total_cost_with_contingencies`, `conductor_cost`, `structure_cost`, `converter_cost`, `conductor_cost_with_contingencies`, `structure_cost_with_contingencies`, `converter_cost_with_contingencies`, `weighted_miles`, `average_terrain_multiplier`. Structure and converter costs are 0 for reconductoring.

Key Logic:

```
1 weighted_miles, average_terrain_multiplier =  
    calculate_weighted_miles()  
2 conductor_cost = (variable_conductor_cost_per_mile * weighted_miles)  
    + fixed_conductor_cost
```

```

3 if reconductoring:
4     structure_cost = 0
5     converter_cost = 0
6 else:
7     structure_cost = variable_structure_cost_per_mile *
8         weighted_miles
9     converter_cost = fixed_converter_cost * number_of_converters
10 total_cost = conductor_cost + structure_cost + converter_cost
11 conductor_cost_with_contingencies = conductor_cost * (1 +
12     contingencies["conductor_contingency"])
13 structure_cost_with_contingencies = structure_cost * (1 +
14     contingencies["structure_contingency"])
15 converter_cost_with_contingencies = converter_cost * (1 +
16     contingencies["converter_contingency"])
17 total_cost_with_contingencies = (conductor_cost_with_contingencies
18     + structure_cost_with_contingencies +
19     converter_cost_with_contingencies)
20 return BuildCosts(...)

```

3.2 main()

Main execution function that orchestrates the build cost calculation and output.

Workflow:

1. Loads project technical details; constructs category via `build_category_string(project_details)`
2. Determines `number_of_converters` from `get_project_data_raw()["project"]["number_of_converters"]` if DC, else 0
3. Loads physical details, contingencies, financing; calls `load_costs(category, total_miles, number_of_converters, contingencies, project_details.reconductoring)`
4. Loads AFUDC setup via `load_afudc_setup()`; if `apply_afudc`, calculates AFUDC capitalized cost via `calculate_afudc_capitalized_cost(...)`
5. Computes `construction_start_year = calculate_construction_start_year(delay_years)`; if `construction_years > 0`, `annual_build_cost = total with contingencies / construction_years`, `build_cost_pv = calculate_present_value(annual_build_cost, wacc_real, construction_years, construction_start_year)`; else one-time PV at `construction_start_year`
6. Builds results dict (`total_nominal`, `total_afudc`, `total_pv`, component nominals and zeros for component AFUDC); `CTCCOutputManager().add_build_costs(results).write_batch(...)`

4 Example

The following example demonstrates a typical usage of the build costs calculation:

```

1 # The script is typically called from ctcc.py, but can be run
  standalone
2 # It automatically loads configuration from YAML files
3
4 # Example: 500 MW Overhead AC line with Advanced Aluminum Conductor
5 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
6 #
7 # Inputs (from YAML):
8 #   - Construction type: Overhead
9 #   - AC/DC: AC
10 #   - Capacity: 500 MW
11 #   - Conductor: Advanced Aluminum
12 #   - Total miles: 100 miles
13 #   - Terrain: 60% flat, 40% forested
14 #   - Contingencies: 15% conductor, 20% structure, 10% converter
15 #
16 # Calculation:
17 #   1. Weighted miles = 100 * (0.6*1.0 + 0.4*1.2) = 108 miles
18 #   2. Conductor cost = ($500k/mile * 108) + $1M = $55M
19 #   3. Structure cost = $200k/mile * 108 = $21.6M
20 #   4. Converter cost = 0 (AC project)
21 #   5. Total base = $76.6M
22 #   6. With contingencies = $76.6M * (1 + weighted avg)      $88M
23 #
24 # Output:
25 #   - Nominal cost: $88M
26 #   - AFUDC capitalized: $92M (if AFUDC enabled)
27 #   - Present value: $88M (discounted at real WACC)

```

Integration Example:

```

1 # When called from ctcc.py:
2 # 1. Script loads project details from YAML
3 # 2. Calculates build costs
4 # 3. Writes to CSV via CTCCOutputManager:
5 csv_manager = CTCCOutputManager()
6 results = {
7     "total_nominal": costs.total_cost_with_contingencies,
8     "total_afudc": capitalized_cost_with_contingencies if
9         afudc_setup.apply_afudc else 0,
10    "total_pv": build_cost_pv,
11    "conductor_nominal": costs.conductor_cost_with_contingencies,
12    "structure_nominal": costs.structure_cost_with_contingencies,
13    "converter_nominal": costs.converter_cost_with_contingencies,
14    "conductor_afudc": 0, "structure_afudc": 0, "converter_afudc":
15        0,
16 }
17 csv_manager.add_build_costs(results)

```

```
16 csv_manager.write_batch_summary()
```

5 Key Implementation Details

5.1 Reconductoring Handling

For reconductoring projects, the script sets structure and converter costs to zero since existing infrastructure is reused. Only conductor costs are calculated.

5.2 AFUDC Calculation

If AFUDC is enabled (`load_afudc_setup().apply_afudc`), the script calculates capitalized costs via `calculate_afudc_capitalized_cost` using the AFUDC rate and cost timing patterns (`afudc_setup.timing_patterns["build_costs"]`). This represents the regulatory perspective where construction financing costs are capitalized.

5.3 Present Value of Build Costs

The script computes present value of build costs spread evenly over the construction period: `annual_build_cost = total_cost_with_contingencies / construction_years`, then `calculate_present_value(annual_build_cost, wacc_real, construction_years, construction_start_year)`. If `construction_years` is 0, build cost is treated as a one-time cost at `construction_start_year`.

5.4 Weighted Miles

The script uses `calculate_weighted_miles()` to adjust costs for terrain complexity. Different terrain types have different cost multipliers, and the weighted average is applied to the total miles.